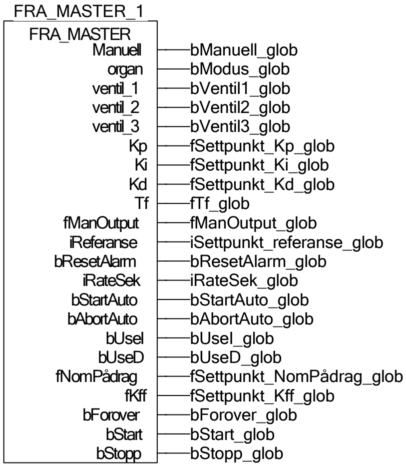
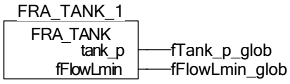
Structured Ladder/FBD 5/3/2026
Data Name : PLS_PROGRAM

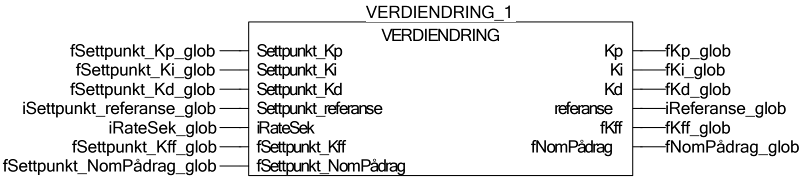
2



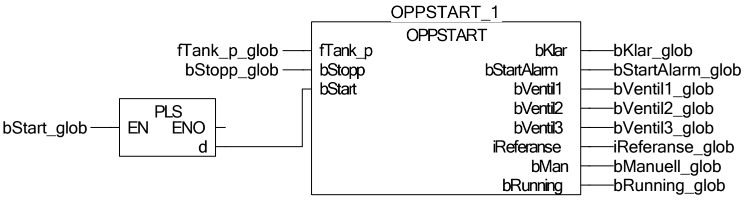
3



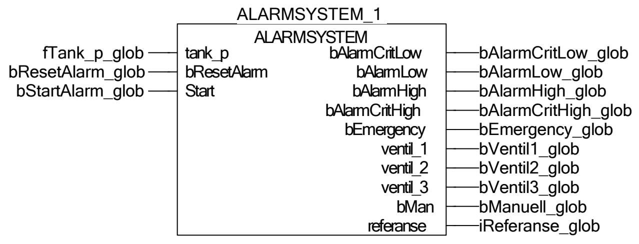
4



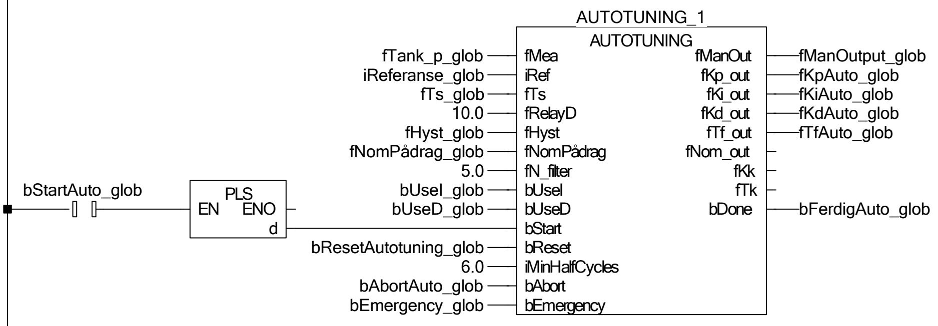
5



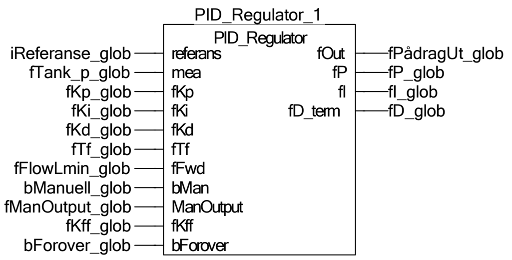
6


ALARMSYSTEM_1
ALARMSYSTEM
fTank_p_glob
bResetAlarm_glob
bStartAlarm_glob
tank_p
bResetAlarm
Start
bAlarmCritLow
bAlarmLow
bAlarmHigh
bAlarmCritHigh
bEmergency
ventil_1
ventil_2
ventil_3
bMan
referanse
bAlarmCritLow_glob
bAlarmLow_glob
bAlarmHigh_glob
bAlarmCritHigh_glob
bEmergency_glob
bVentil1_glob
bVentil2_glob
bVentil3_glob
bManuell_glob
iReferanse_glob

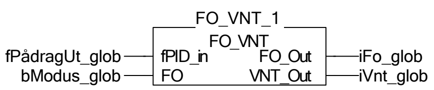
7


AUTOTUNING_1
AUTOTUNING
bStartAuto_glob
PLS
EN
ENO
d
fTank_p_glob
iReferanse_glob
fTs_glob
10.0
fHyst_glob
fNomPådrag_glob
5.0
bUseI_glob
bUseD_glob
bResetAutotuning_glob
6.0
bAbortAuto_glob
bEmergency_glob
fMea
iRef
fTs
fRelayD
fHyst
fNomPådrag
fN_filter
bUseI
bUseD
bStart
bReset
iMinHalfCycles
bAbort
bEmergency
fManOut
fKp_out
fKi_out
fKd_out
fTf_out
fNom_out
fKk
fTk
bDone
fManOutput_glob
fKpAuto_glob
fKiAuto_glob
fKdAuto_glob
fTfAuto_glob
bFerdigAuto_glob

8


PID_Regulator_1
PID_Regulator
iReferanse_glob
fTank_p_glob
fKp_glob
fKi_glob
fKd_glob
fTf_glob
fFlowLmin_glob
bManuell_glob
fManOutput_glob
fKff_glob
bForover_glob
referans
mea
fKp
fKi
fKd
fTf
fFwd
bMan
ManOutput
fKff
bForover
fOut
fP
fI
fD_term
fPådragUt_glob
fP_glob
fI_glob
fD_glob

9


FO_VNT_1
FO_VNT
fPådragUt_glob
bModus_glob
fPID_in
FO
FO_Out
VNT_Out
iFo_glob
iVnt_glob

10

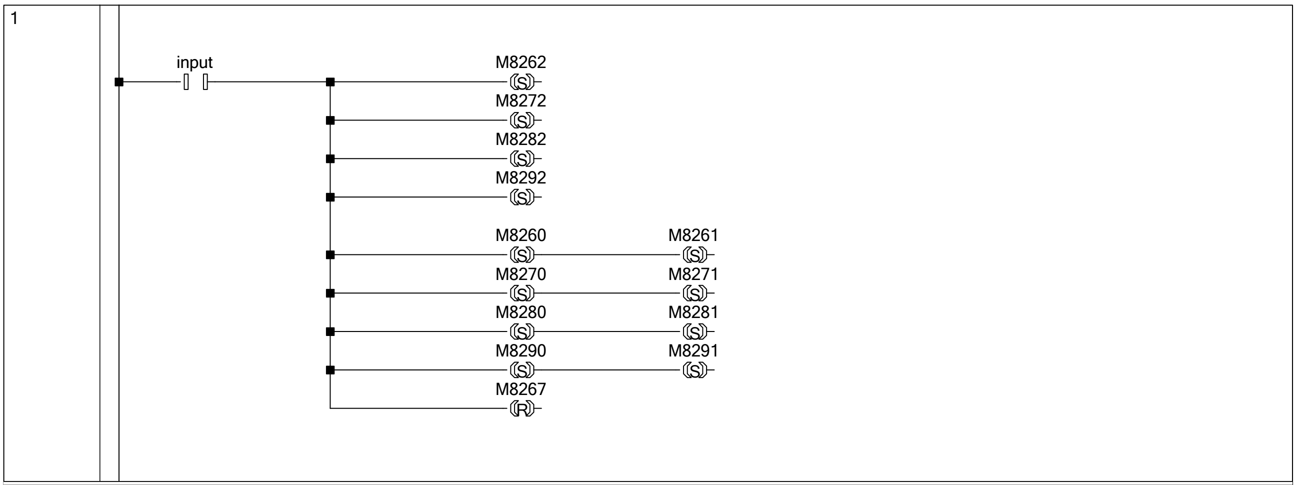
TIL_TANK_1

TIL_TANK	
1	input
iVnt_glob	vnt_flow
iFo_glob	fo_flow

11

TIL_MASTER_1

TIL_MASTER	
fTank_p_glob	tank_p
bAlarmCritLow_glob	bAlarmCritLow
bAlarmLow_glob	bAlarmLow
bAlarmHigh_glob	bAlarmHigh
bAlarmCritHigh_glob	bAlarmCritHigh
bEmergency_glob	bEmergency
fPådragUt_glob	fPådragUt
fP_glob	fP
fI_glob	fI
fD_glob	fD_term
iFo_glob	fo_flow
fKpAuto_glob	auto_p
fKiAuto_glob	auto_i
fKdAuto_glob	auto_d
fTfAuto_glob	auto_tf
bRunning_glob	bRunning
bKlar_glob	bKlar
fFlowLmin_glob	fFlowLmin


1
input
M8262
M8272
M8282
M8292
M8260
M8261
M8270
M8271
M8280
M8281
M8290
M8291
M8267

```
IF Start THEN
   IF bStopp_glob THEN
      bEmergency := FALSE;
      bLowEmergency := FALSE;
      bAlarmFerdig_glob := FALSE;
      bAlarmCritLow := FALSE;
      bAlarmCritHigh := FALSE;
      bAlarmLow := FALSE;
      bAlarmHigh := FALSE;
      timer := FALSE;
      RETURN;
   END_IF;

   bAlarmCritLow := tank_p < 10.0;
   bAlarmLow := tank_p < 20.0 AND NOT bAlarmCritLow;
   bAlarmHigh := tank_p > 80.0 AND NOT bAlarmCritHigh;
   bAlarmCritHigh := tank_p > 90.0;


   IF bAlarmLow OR bAlarmHigh THEN
      bLowEmergency := TRUE;
   END_IF;

  IF bAlarmCritHigh OR bAlarmCritLow THEN
      bEmergency := TRUE;
      bAlarmFerdig_glob := FALSE;
      ventil_1 := TRUE;
      ventil_2 := TRUE;
      ventil_3 := FALSE;
      referanse := 50;
      bMan := FALSE;
   END_IF;

   IF bEmergency THEN
      IF NOT bAlarmFerdig_glob THEN
         ventil_1 := TRUE;
         ventil_2 := TRUE;
         ventil_3 := FALSE;
         referanse := 50;
         bMan := FALSE;

         IF tank_p > 45.0 AND tank_p < 55.0 THEN
            timer := TRUE;
            OUT_T(timer, TC0, 50);
            IF TS0 THEN
               bAlarmFerdig_glob := TRUE;
            END_IF;
         ELSE
            timer := FALSE;
         END_IF;
         ELSE
            IF NOT bResetAlarm THEN
               bMan := TRUE;
               fManOutput_glob := 0.0;
               ventil_1 := FALSE;
               ventil_2 := FALSE;
               ventil_3 := FALSE;
               referanse := 50;
            END_IF;
         END_IF;
      END_IF;
   END_IF;

IF bResetAlarm AND bAlarmFerdig_glob AND tank_p > 45.0 AND tank_p < 55.0 THEN
      bEmergency := FALSE;
      bLowEmergency := FALSE;
      bAlarmCritLow := FALSE;
      bAlarmCritHigh := FALSE;
      bAlarmFerdig_glob := FALSE;
      bMan := FALSE;
      ventil_1 := TRUE;
      ventil_2 := TRUE;
      ventil_3 := FALSE;
END_IF;

IF bResetAlarm THEN
   bLowEmergency := FALSE;
END_IF;

(* alarmlampe *)
IF NOT bResetAlarm THEN
   IF bEmergency OR bLowEmergency THEN
      Y1 := M8013;
   ELSE
      Y1 := FALSE;
   END_IF;
END_IF;
```

```
FLT(TRUE, iRef, fRef);
IF M8002 THEN
   fHyst_glob:= 1.5;
END_IF;

(* Reset *)
IF bReset THEN
   fError        := 0.0;
   fError_prev    := 0.0;
   fPeakHigh      := -1.0E+30;
   fPeakLow       := 1.0E+30;
   fAmplitude     := 0.0;
   fPeriod        := 0.0;
   fLastCrossTime := 0.0;
   fCycleTimer    := 0.0;
   iHalfCycles    := 0.0;
   bRelayState    := FALSE;
   bInitDone      := FALSE;
   bRunning       := FALSE;
   bDone          := FALSE;
   bStartLatch    := FALSE;
   fManOut        := fNomPådrag;
   fKp_out        := 0.0;
   fKi_out        := 0.0;
   fKd_out        := 0.0;
   fTf_out        := 0.0;
   fNom_out       := 0.0;
   fKk        := 0.0;
   fTk        := 0.0;
   RETURN;
END_IF;

(* Latch start-puls *)
IF bStart THEN
   bStartLatch := TRUE;
END_IF;

(* Start-flanke: initialiser *)
IF bStartLatch AND (NOT bInitDone) AND (NOT bDone) THEN
   bWasManual     := bManuell_glob;

   fPeakHigh      := -1.0E+30;
   fPeakLow       := 1.0E+30;
   fCycleTimer    := 0.0;
   fLastCrossTime := 0.0;
   iHalfCycles    := 0.0;
   fError_prev    := 0.0;
   bRelayState    := FALSE;
   bRunning       := TRUE;
   bDone          := FALSE;
   bInitDone      := TRUE;
   fManOut        := fNomPådrag;
   bStartLatch    := FALSE;
END_IF;

(* Hoveddel *)
IF bRunning THEN

   bManuell_glob   := TRUE;
   fManOutput_glob := fManOut;

   IF bAbort OR bEmergency THEN
      bRunning      := FALSE;
      bDone         := FALSE;
      bInitDone     := FALSE;
      bManuell_glob := bWasManual;
      fManOut       := fNomPådrag;
      RETURN;
   END_IF;

   fError := fRef - fMea;
   fCycleTimer := fCycleTimer + fTs;

   IF fError > fHyst THEN
      bRelayState := TRUE;
   ELSIF fError < -fHyst THEN
      bRelayState := FALSE;
   END_IF;

   IF bRelayState THEN
      fManOut := fNomPådrag + fRelayD;
   ELSE
      fManOut := fNomPådrag - fRelayD;
   END_IF;

   IF (fError * fError_prev < 0.0) THEN

      IF iHalfCycles >= iMinHalfCycles THEN
         fAmplitude := (fPeakHigh - fPeakLow) / 2.0;
```

```
        IF fAmplitude > 0.0 THEN
            (* Kritisk forsterkning og kritisk periodetid *)
            fKk := (4.0 * fRelayD) / (3.14159 * fAmplitude);
            fTk := fPeriod;

            (* P *)
            IF (NOT bUseI) AND (NOT bUseD) THEN
                fKp_out  := 0.5 * fKk;
                fKi_out  := 0.0;
                fKd_out  := 0.0;
                fTf_out  := 0.0;
                fNom_out := fNomPådrag;

            (* PI – Tyreus-Luyben *)
            ELSIF bUseI AND (NOT bUseD) THEN
                fKp_out  := fKk / 3.2;
                fKi_out  := fKp_out / (2.2 * fTk);
                fKd_out  := 0.0;
                fTf_out  := 0.0;
                fNom_out := 0.0;

            (* PD *)
            ELSIF (NOT bUseI) AND bUseD THEN
                fKp_out  := 0.5 * fKk;
                fKi_out  := 0.0;
                fTd_temp := fTk / 8.0;
                fKd_out  := fKp_out * fTd_temp;
                fNom_out := fNomPådrag;
                IF fN_filter > 0.0 THEN
                    fTf_out := fTd_temp / fN_filter;
                ELSE
                    fTf_out := 0.0;
                END_IF;

            (* PID –C *)
            ELSIF bUseI AND bUseD THEN
                fKp_out  := fKk / 3.2;
                fKi_out  := fKp_out / (2.2 * fTk);
                fTd_temp := fTk/ 6.3;
                fKd_out  := fKp_out * fTd_temp;
                fNom_out := 0.0;
                IF fN_filter > 0.0 THEN
                    fTf_out := fTd_temp / fN_filter;
                ELSE
                    fTf_out := 0.0;
                END_IF;
            END_IF;
        END_IF;

        bDone          := TRUE;
        bRunning       := FALSE;
        bManuell_glob  := 0;
        fManOutput_glob := fNomPådrag;
        fManOut        := fNomPådrag;
    END_IF;

    IF iHalfCycles > 0.0 THEN
        fPeriod := 2.0 * (fCycleTimer - fLastCrossTime);
    END_IF;
    fLastCrossTime := fCycleTimer;
    iHalfCycles := iHalfCycles + 1.0;

    fPeakHigh := -1.0E+30;
    fPeakLow  := 1.0E+30;
  END_IF;

  IF fMea > fPeakHigh THEN fPeakHigh := fMea; END_IF;
  IF fMea < fPeakLow  THEN fPeakLow  := fMea; END_IF;

  fError_prev := fError;
END_IF;
```

```
INT(input, fPID_in, iPID_in);

IF NOT FO AND FO_prev THEN
   bTransisjon := TRUE;
   iFO_hold := FO_Out;
END_IF;
FO_prev := FO;

OUT_T(bTransisjon, TC6,40);
IF TS6 THEN
   bTransisjon := FALSE;
END_IF;

IF bTransisjon THEN
   FO_Out := iFO_hold;
   VNT_Out := iPID_in * 45;
END_IF;

IF NOT bTransisjon THEN
   IF FO THEN
      FO_Out := REAL_TO_INT(7000.0 + (fPID_in/100.0) * (16384.0 - 7000.0));
      VNT_Out := 4000;
   ELSE
      FO_Out := 16384;
      VNT_Out := iPID_in * 45;
   END_IF;
END_IF;

IF bStopp_glob OR bAlarmFerdig_glob THEN
   FO_Out := 0;
   VNT_Out := 0;
END_IF;
```

```
(* ==================== FRA MASTER ==================== *)

(*  Referanse *)
FROM(TRUE, 0, 0, 1, D10);

IF iReferanse<> D10 THEN
   iReferanse:= D10;
END_IF;

(* Ventiler *)
FROM(TRUE, 0, 1, 1, D11);
ventil_1   := D11.0;
ventil_2   := D11.1;
ventil_3   := D11.2;

(* PID-parametere*)

(* Kp *)
FROM(TRUE, 0, 4, 1, D14);
FLT(TRUE, D14, fKp_hmi);
fKp_hmi := fKp_hmi/1000.0;
IF bEmergency_glob OR (bRunning_glob AND NOT bKlar_glob) THEN
   Kp := fSafeKp;
ELSE
   Kp := fKp_hmi;
END_IF;

(* Ki *)
FROM(TRUE, 0, 5, 1, D16);
FLT(TRUE, D16, fKi_hmi);
fKi_hmi := fKi_hmi/1000.0;
IF bEmergency_glob OR (bRunning_glob AND NOT bKlar_glob) THEN
   Ki := fSafeKi;
ELSE
   Ki := fKi_hmi;
END_IF;

(* Kd *)
FROM(TRUE, 0, 6, 1, D18);
FLT(TRUE, D18, fKd_hmi);
fKd_hmi := fKd_hmi/1000.0;
IF bEmergency_glob OR (bRunning_glob AND NOT bKlar_glob) THEN
   Kd := fSafeKd;
ELSE
   Kd := fKd_hmi;
END_IF;

(* Tf *)
FROM(TRUE, 0, 8, 1, D20);
FLT(TRUE, D20, fTf_hmi);
fTf_hmi := fTf_hmi/1000.0;
IF bEmergency_glob OR (bRunning_glob AND NOT bKlar_glob) THEN
   Tf := fSafeTf;
ELSE
   Tf := fTf_hmi;
END_IF;

IF fKp_hmi = 0.5 AND fKi_hmi = 0.1 AND NOT bEmergency_glob AND NOT (bRunning_glob AND NOT bKlar_glob) THEN
   Kp := fSafeKp;
   Ki := 0.05;
   Kd := fKd_hmi;
   Tf := fTf_hmi;
ELSE
   Kp := fKp_hmi;
   Ki := fKi_hmi;
   Kd := fKd_hmi;
   Tf := fTf_hmi;
END_IF;



(* Manuelt pådrag *)
FROM(TRUE, 0, 7, 1, D22);
FLT(TRUE, D22, fManOutput);
fManOutput := fManOutput/100.0;

(* Organ *)
Organ      := D11.3;

(* manuell eller auto *)
Manuell := D11.4;
bManuell_glob := Manuell;

(* Reset alarm *)
bResetAlarm := D11.5;

(* Bruk foroverkobling *)
bForover := D11.6;

(* Start/stopp *)
```

```
bStart := D11.7;
bStopp := D11.8;



(* Autotuning *)
FROM(TRUE, 0, 12, 1, D28);

bStartAuto  := D28.0;
bAbortAuto := D28.4;

IF D28.5 THEN bResetAutotuning_glob := TRUE; ELSE bResetAutotuning_glob := FALSE; END_IF;


IF D28.2 THEN
   bUseI := TRUE;
ELSE
   bUseI := FALSE;
END_IF;

IF D28.3 THEN
   bUseD := TRUE;
ELSE
   bUseD := FALSE;
END_IF;


(* Nom pådrag *)
FROM(TRUE, 0, 13, 1, D29);
FLT(TRUE, D29, fNomPådrag);
fNomPådrag := fNomPådrag/100.0;

(* Foroverkoblingskoeffisient *)
FROM(TRUE, 0, 14, 1, D30);
FLT(TRUE, D30, fKff);
fKff:= fKff/100.0;

(* Ratebegrensning *)
FROM(TRUE, 0, 15, 1, D31);
iRateSek := D31;
```

```
MOV(TRUE, D8260, tank);
FLT(TRUE, tank, ftank);

IF ftank >= 500.0 AND ftank <= 1567.0 THEN
    fLastValid := ftank;
END_IF;

tank_p_raw := ((fLastValid - 500.0) * 100.0) / (1567.0 - 500.0);

fAlpha := 0.02;
IF NOT bFirstRun THEN
    tank_p_filt := tank_p_raw;
    bFirstRun := TRUE;
ELSE
    tank_p_filt := fAlpha * tank_p_raw + (1.0 - fAlpha) * tank_p_filt;
END_IF;
tank_p := tank_p_filt;

FLT(TRUE, D8261, fFlow_hmi);
fFlowLmin := ((fFlow_hmi - 400.0) / (1800.0 - 400.0)) * 6.6;
IF fFlowLmin < 0.0 THEN fFlowLmin := 0.0; END_IF;
```

Function Block

```
Temp32 := INT_TO_DINT(rInn - offset) * 100 / INT_TO_DINT(maksverdi);
ProsentUt:= DINT_TO_INT(Temp32);
```

```
IF bStopp THEN
   bKlar := FALSE;
   bRunning := FALSE;
   bStartAlarm := FALSE;
   bVentil1 := FALSE;
   bVentil2 := FALSE;
   bVentil3 := FALSE;
   bMan := TRUE;
   fManOutput_glob := 0.0;
   fPådragUt_glob := 0.0;
   bStartupTimer := FALSE;
   RETURN;
END_IF;

IF bStart AND NOT bRunning THEN
   bRunning      := TRUE;
   bKlar         := FALSE;
   bStartupTimer := FALSE;
   bResetAlarm_glob := TRUE;
END_IF;

IF bRunning THEN
   IF NOT bKlar THEN
      bStartAlarm := FALSE;
      bVentil1    := TRUE;
      bVentil2    := TRUE;
      bVentil3    := FALSE;

      iReferanse  := 50;
      bMan        := FALSE;

      IF fTank_p > 45.0 AND fTank_p < 55.0 THEN
         bStartupTimer := TRUE;
      ELSE
         bStartupTimer := FALSE;
      END_IF;

      OUT_T(bStartupTimer, TC1, 50);

      IF TS1 THEN
         bKlar         := TRUE;
         bStartAlarm   := TRUE;
         bRunning      := FALSE;
         bStartupTimer := FALSE;
      END_IF;

   ELSE
      bStartAlarm := TRUE;
   END_IF;
END_IF;

MOV(TRUE, D8010, iSkanTid);
FLT(TRUE, iSkanTid, fSkanTid);
fTs_glob := fSkanTid/10000.0;
```

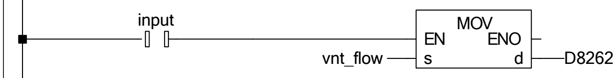
```
FLT(input,referans,fRef);
  fTs := fTs_glob;
  bUseI := fKi > 0.0;
  bUseD := fKd > 0.0;
  fError := fRef - mea;
  IF bMan AND NOT bMan_prev THEN
     fManOutput_glob := fOut;
  END_IF;
  bMan_prev := bMan;
  (* P-ledd *)
  fP := fKp * fError;
(* I-ledd *)
IF bUseI THEN
   bRampNomActive := FALSE;
   IF (NOT bUseI_prev) AND bInitDone THEN
      (* Bumpless P/PD -> PI/PID: forover trekkes bare fra hvis den faktisk er på *)
      IF bForover THEN
         fI := fOut - fP - fD_term - (fFwd * fKff);
      ELSE
         fI := fOut - fP - fD_term;
      END_IF;
   ELSE
      fI := fI + fTs * (fKi * fError + fTracking);
      IF fI > fOutMax THEN fI := fOutMax; END_IF;
      IF fI < fOutMin THEN fI := fOutMin; END_IF;
   END_IF;
ELSE
   (* Fallende flanke: start ramp fra nåværende fI ned til fNomPådrag_glob *)
   IF bUseI_prev THEN
      fI_rampStart    := fI;
      fI_rampProgress := 0.0;
      bRampNomActive  := TRUE;
   END_IF;

   IF bRampNomActive THEN
      fI_rampProgress := fI_rampProgress + fTs / 3.0;
      IF fI_rampProgress >= 1.0 THEN
         fI_rampProgress := 1.0;
         bRampNomActive  := FALSE;
      END_IF;
      (* Smoothstep 3t² - 2t³ - samme som i Rampe-blokken din *)
      fT_smooth := fI_rampProgress * fI_rampProgress * (3.0 - 2.0 * fI_rampProgress);
      fI := fI_rampStart + (fNomPådrag_glob - fI_rampStart) * fT_smooth;
   ELSE
      fI := fNomPådrag_glob;
   END_IF;
END_IF;
bUseI_prev := bUseI;
bInitDone  := TRUE;
(* D-ledd - derivasjon på måling, ikke feil (unngår derivative kick) *)
IF bUseD AND fTf > 0.0 THEN
   fDfilter := fDfilter + fTs / fTf * (fKd / fTs * (fMea_prev - mea) - fDfilter);
   fD_term := fDfilter;
ELSE
   fD_term := 0.0;
END_IF;  (* Foroverkobling *)
  fhmiOut := fP + fI + fD_term;
  IF bForover THEN
     fhmiOut := fhmiOut + (fFwd * fKff);
  END_IF;
  (* Gulvverdi *)
  fLocalMin := fNomPådrag_glob;
  (* Flagg: stort sprang ned *)
  IF (iRef_prev - iSettpunkt_referanse_glob) > 15 THEN
     bLargeDownStep := TRUE;
  END_IF;
  iRef_prev := iSettpunkt_referanse_glob;
  (* Gulv + I-tracking under stort sprang ned *)
  IF bLargeDownStep AND mea < (INT_TO_REAL(iSettpunkt_referanse_glob) + 5.0) THEN
     IF fhmiOut < fLocalMin THEN
        fhmiOut := fLocalMin;
     END_IF;
     fI := fhmiOut - fP - fD_term;
     IF fI > fOutMax THEN fI := fOutMax; END_IF;
     IF fI < fOutMin THEN fI := fOutMin; END_IF;
     bLargeDownStep := FALSE;
  END_IF;
  IF bMan THEN
     fOut := ManOutput;
     IF bForover THEN
        fI := ManOutput - fP - fD_term - (fFwd * fKff);
     ELSE
        fI := ManOutput - fP - fD_term;
     END_IF;
  ELSE
     IF fhmiOut > fOutMax THEN
        fOut    := fOutMax;
        bSaturated := TRUE;
     ELSIF fhmiOut < fOutMin THEN
        fOut    := fOutMin;
```

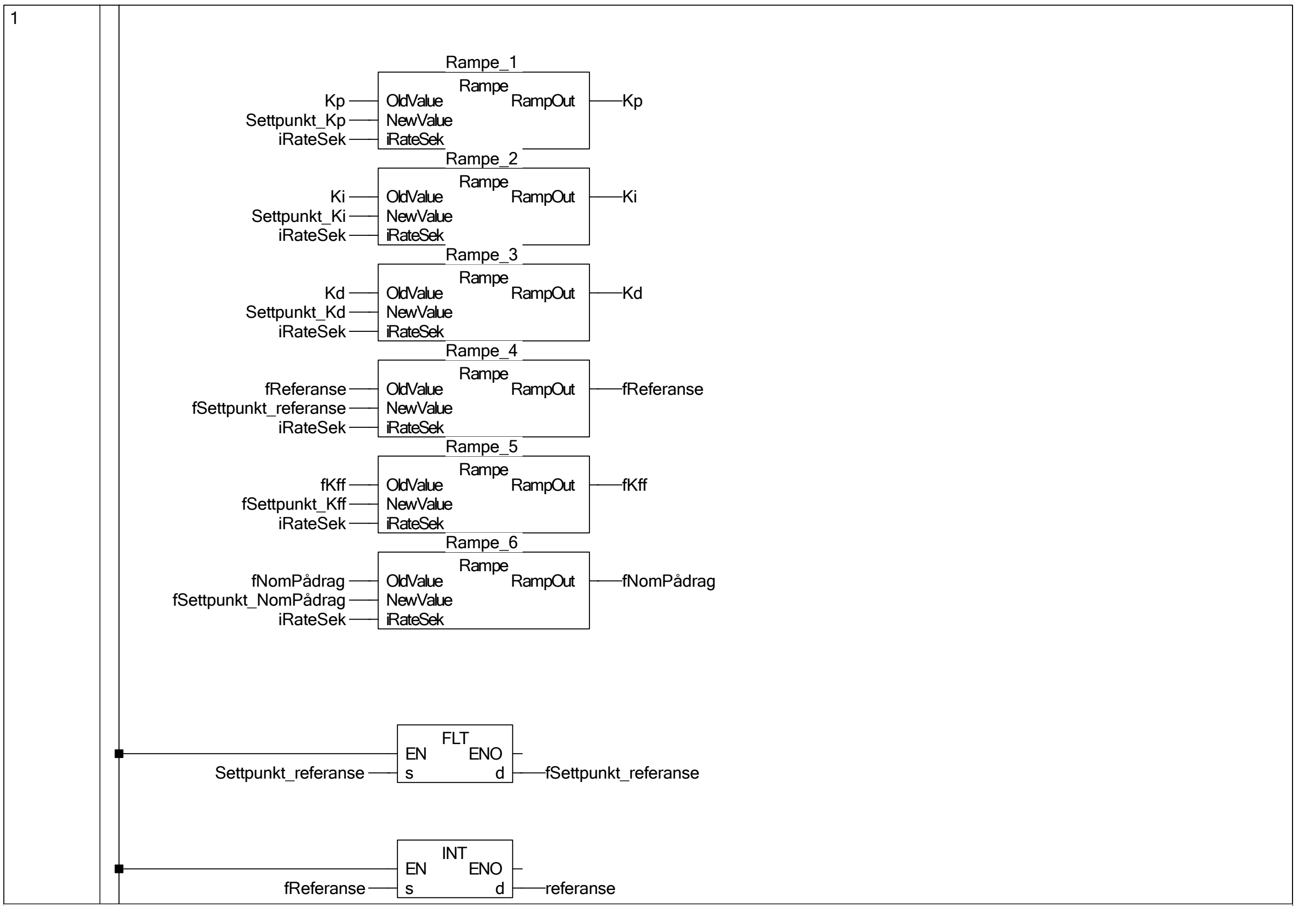
```
        bSaturated := TRUE;
    ELSE
        fOut    := fhmiOut;
        bSaturated := FALSE;
    END_IF;
    IF fKp > 0.0 THEN
        fKt := (fKi / fKp) * 1.5;
    ELSE
        fKt := 0.0;
    END_IF;
    fTracking := fKt * (fOut - fhmiOut);
END_IF;
fError_prev := fError;
 fMea_prev := mea;
```

```
fTs := fTs_glob;
FLT(TRUE, iRateSek, fRateSek);
IF fRateSek > 0.0 THEN
   RampStep := fTs / fRateSek;
END_IF;

IF NewValue <> rLastTarget THEN
   rStartValue  := RampOut;
   rProgress    := 0.0;
   rLastTarget  := NewValue;
END_IF;

IF rProgress < 1.0 THEN
   rProgress := rProgress + RampStep;

   IF rProgress > 1.0 THEN
      rProgress := 1.0;
   END_IF;

   (*Smoothstep funksjon, formel: 3t² - 2t³ *)
   rT_smooth := rProgress * rProgress * (3.0 - (2.0 * rProgress));

   RampOut := rStartValue + ((NewValue - rStartValue) * rT_smooth);
ELSE
   RampOut := NewValue;
END_IF;
```

```
(* ===================== TIL MASTER ===================== *)

(* Tank *)
INT(TRUE, tank_p*100.0, D23);
TO(TRUE, D23, 0, 10, 1);

(* Pådrag *)
INT(TRUE, fPådragUt, D24);
TO(TRUE, D24, 0, 2, 1);

(* Frekvensomformer pådrag *)

TO(TRUE, fo_flow, 0, 3, 1);

(* Pådrag Kp *)
INT(TRUE, fP*100.0, iP);
TO(TRUE, iP, 0, 4, 1);

(* Pådrag Ki *)
INT(TRUE, fI*100.0, iI);
TO(TRUE, iI, 0, 5, 1);

(* Pådrag Kd *)
INT(TRUE, fD_term*100.0, iD);
TO(TRUE, iD, 0, 6, 1);

(* Alarmer *)
iAlarmWord := 0;
IF bAlarmCritLow THEN iAlarmWord := iAlarmWord + 1; END_IF;
IF bAlarmLow THEN iAlarmWord := iAlarmWord + 2; END_IF;
IF bAlarmCritHigh THEN iAlarmWord := iAlarmWord + 4; END_IF;
IF bAlarmHigh THEN iAlarmWord := iAlarmWord + 8; END_IF;
IF bEmergency THEN iAlarmWord := iAlarmWord + 16; END_IF;
IF bManuell_glob THEN iAlarmWord := iAlarmWord + 32; END_IF;
IF bAlarmFerdig_glob THEN iAlarmWord := iAlarmWord + 64; END_IF;
IF bRunning_glob THEN iAlarmWord := iAlarmWord + 128; END_IF;
IF bVentil1_glob THEN iAlarmWord := iAlarmWord + 256; END_IF;
IF bVentil2_glob THEN iAlarmWord := iAlarmWord + 512; END_IF;
IF bVentil3_glob THEN iAlarmWord := iAlarmWord + 1024; END_IF;
TO(TRUE, iAlarmWord, 0, 7, 1);

(* Autotuning *)
IF bFerdigAuto_glob THEN
   INT(TRUE, auto_p*1000.0, iauto_p);
   TO(TRUE, iauto_p, 0, 11, 1);

   INT(TRUE, auto_i*1000.0, iauto_i);
   TO(TRUE, iauto_i, 0, 12, 1);

   INT(TRUE,auto_d*1000.0, iauto_d);
   TO(TRUE, iauto_d, 0, 13, 1);

   INT(TRUE,auto_tf*1000.0, iauto_tf);
   TO(TRUE, iauto_tf, 0, 14, 1);
END_IF;

TO(TRUE, iReferanse_glob, 0, 0, 1);


(* Oppstart *)

iStatus := 0;
IF bRunning AND NOT bKlar THEN iStatus := 1; END_IF;
IF NOT bRunning AND bKlar    THEN iStatus := 2; END_IF;

TO(TRUE, iStatus, 0, 15, 1);

(* Utstrømsmåling *)
INT(TRUE, fFlowLmin*1000.0, iFlowLmin);
TO(TRUE, iFlowLmin, 0, 1, 1);
```

1




1
Rampe_1
Rampe
Kp OldValue RampOut Kp
Settpunkt_Kp NewValue
iRateSek iRateSek
Rampe_2
Rampe
Ki OldValue RampOut Ki
Settpunkt_Ki NewValue
iRateSek iRateSek
Rampe_3
Rampe
Kd OldValue RampOut Kd
Settpunkt_Kd NewValue
iRateSek iRateSek
Rampe_4
Rampe
fReferanse OldValue RampOut fReferanse
fSettpunkt_referanse NewValue
iRateSek iRateSek
Rampe_5
Rampe
fKff OldValue RampOut fKff
fSettpunkt_Kff NewValue
iRateSek iRateSek
Rampe_6
Rampe
fNomPådrag OldValue RampOut fNomPådrag
fSettpunkt_NomPådrag NewValue
iRateSek iRateSek
FLT
EN ENO
Settpunkt_referanse s d fSettpunkt_referanse
INT
EN ENO
fReferanse s d referanse

Label
Data Name : Global1
Global Label Setting

5/3/2026

	Class	Label Name	Data Type	Constant	Device	Address	Comment	Remark	Relation with System Label	System Label Name	Attribute
1	VAR_GLOBAL	iSettpunkt_referanse_glob	Word[Signed]								
2	VAR_GLOBAL	iReferanse_glob	Word[Signed]								
3	VAR_GLOBAL	bManuell_glob	Bit								
4	VAR_GLOBAL	bModus_glob	Bit								
5	VAR_GLOBAL	fManOutput_glob	FLOAT (Single Precision)								
6	VAR_GLOBAL	fRampStep_glob	FLOAT (Single Precision)								
7	VAR_GLOBAL	bForover_glob	Bit								
8	VAR_GLOBAL	PID_Regulator_1	PID_Regulator								
9	VAR_GLOBAL	VERDIENDRING_1	VERDIENDRING								
10	VAR_GLOBAL	fSettpunkt_Kp_glob	FLOAT (Single Precision)								
11	VAR_GLOBAL	fSettpunkt_Ki_glob	FLOAT (Single Precision)								
12	VAR_GLOBAL	fSettpunkt_Kd_glob	FLOAT (Single Precision)								
13	VAR_GLOBAL	fKp_glob	FLOAT (Single Precision)								
14	VAR_GLOBAL	fKi_glob	FLOAT (Single Precision)								
15	VAR_GLOBAL	fKd_glob	FLOAT (Single Precision)								
16	VAR_GLOBAL	fTf_glob	FLOAT (Single Precision)								
17	VAR_GLOBAL	fNomPådrag_glob	FLOAT (Single Precision)								
18	VAR_GLOBAL	fKff_glob	FLOAT (Single Precision)								
19	VAR_GLOBAL	bUseI_glob	Bit								
20	VAR_GLOBAL	bUseD_glob	Bit								
21	VAR_GLOBAL	fP_glob	FLOAT (Single Precision)								
22	VAR_GLOBAL	fI_glob	FLOAT (Single Precision)								
23	VAR_GLOBAL	fD_glob	FLOAT (Single Precision)								
24	VAR_GLOBAL	fFlow_p_glob	FLOAT (Single Precision)								
25	VAR_GLOBAL	bRunning_glob	Bit								
26	VAR_GLOBAL	FO_VNT_1	FO_VNT								
27	VAR_GLOBAL	iFo_glob	Word[Signed]								
28	VAR_GLOBAL	iVnt_glob	Word[Signed]		D1	%MW0.1					
29	VAR_GLOBAL	iFlow_glob	Word[Signed]								
30	VAR_GLOBAL	fPådragUt_glob	FLOAT (Single Precision)								
31	VAR_GLOBAL	fTank_p_glob	FLOAT (Single Precision)								
32	VAR_GLOBAL	fFlowLmin_glob	FLOAT (Single Precision)								
33	VAR_GLOBAL	fSettpunkt_NomPådrag_glob	FLOAT (Single Precision)								
34	VAR_GLOBAL	ALARMSYSTEM_1	ALARMSYSTEM								
35	VAR_GLOBAL	bResetAlarm_glob	Bit								
36	VAR_GLOBAL	bAlarmCritLow_glob	Bit								
37	VAR_GLOBAL	bAlarmLow_glob	Bit								
38	VAR_GLOBAL	bAlarmHigh_glob	Bit								
39	VAR_GLOBAL	bAlarmCritHigh_glob	Bit								
40	VAR_GLOBAL	bEmergency_glob	Bit								
41	VAR_GLOBAL	bStartAlarm_glob	Bit								
42	VAR_GLOBAL	bVentil1_glob	Bit		Y004	%QX4					
43	VAR_GLOBAL	bVentil2_glob	Bit		Y005	%QX5					
44	VAR_GLOBAL	bVentil3_glob	Bit		Y006	%QX6					
45											
46	VAR_GLOBAL	AUTOTUNING_1	AUTOTUNING								
47	VAR_GLOBAL	bStartAuto_glob	Bit								
48	VAR_GLOBAL	bResetAutotuning_glob	Bit								
49	VAR_GLOBAL	bAbortAuto_glob	Bit								
50	VAR_GLOBAL	bFerdigAuto_glob	Bit								
51	VAR_GLOBAL	fHyst_glob	FLOAT (Single Precision)								
52	VAR_GLOBAL	fKpAuto_glob	FLOAT (Single Precision)								
53	VAR_GLOBAL	fKiAuto_glob	FLOAT (Single Precision)								
54	VAR_GLOBAL	fKdAuto_glob	FLOAT (Single Precision)								
55	VAR_GLOBAL	fTfAuto_glob	FLOAT (Single Precision)								
56											
57	VAR_GLOBAL	ADDA_initiering_1	ADDA_initiering								
58	VAR_GLOBAL	OPPSTART_1	OPPSTART								
59	VAR_GLOBAL	FRA_MASTER_1	FRA_MASTER								
60	VAR_GLOBAL	FRA_TANK_1	FRA_TANK								
61	VAR_GLOBAL	TIL_MASTER_1	TIL_MASTER								
62	VAR_GLOBAL	TIL_TANK_1	TIL_TANK								
63											
64	VAR_GLOBAL	iRateSek_glob	Word[Signed]								
65											
66	VAR_GLOBAL	bStart_glob	Bit								
67	VAR_GLOBAL	bStopp_glob	Bit								
68	VAR_GLOBAL	fTs_glob	FLOAT (Single Precision)								
69	VAR_GLOBAL	bAlarmFerdig_glob	Bit								
70	VAR_GLOBAL	fSettpunkt_Kff_glob	FLOAT (Single Precision)								
71	VAR_GLOBAL	bKlar_glob	Bit								
72	VAR_GLOBAL_CONSTANT	fSafeKp	FLOAT (Single Precision)	1.2							
73	VAR_GLOBAL_CONSTANT	fSafeKi	FLOAT (Single Precision)	0.1							
74	VAR_GLOBAL_CONSTANT	fSafeKd	FLOAT (Single Precision)	0.8							
75	VAR_GLOBAL_CONSTANT	fSafeTf	FLOAT (Single Precision)	0.5							

Label
Data Name : PLS_PROGRAM
Local Label Setting

5/3/2026

	Class	Label Name	Data Type	Constant	Device	Address	Comment

Label
Data Name : ADDA_initiering
Function/FB Label Setting

5/3/2026

	Class	Label Name	Data Type	Constant	Comment
1	VAR_INPUT	input	Bit		

Label
Data Name : ALARMSYSTEM
Function/FB Label Setting

5/3/2026

	Class	Label Name	Data Type	Constant	Comment
1	VAR_INPUT	tank_p	FLOAT (Single Precision)		
2	VAR_INPUT	bResetAlarm	Bit		
3	VAR_OUTPUT	bAlarmCritLow	Bit		
4	VAR_OUTPUT	bAlarmLow	Bit		
5	VAR_OUTPUT	bAlarmHigh	Bit		
6	VAR_OUTPUT	bAlarmCritHigh	Bit		
7	VAR_OUTPUT	bEmergency	Bit		
8	VAR_OUTPUT	ventil_1	Bit		
9	VAR_OUTPUT	ventil_2	Bit		
10	VAR_OUTPUT	ventil_3	Bit		
11	VAR_OUTPUT	bMan	Bit		
12					
13	VAR_OUTPUT	referanse	Word[Signed]		
14	VAR	timer	Bit		
15	VAR_INPUT	Start	Bit		
16	VAR	bLowEmergency	Bit		
17					
18					
19					
20					
21	VAR	bResetPuls	Bit		
22	VAR	bResetAlarm_prev	Bit		
23	VAR	fKp_saved	FLOAT (Single Precision)		
24	VAR	fKi_saved	FLOAT (Single Precision)		
25	VAR	fKd_saved	FLOAT (Single Precision)		
26	VAR	fTf_saved	FLOAT (Single Precision)		

Label

Data Name : AUTOTUNING

Function/FB Label Setting

5/3/2026

	Class	Label Name	Data Type	Constant	Comment
1	VAR_INPUT	fMea	FLOAT (Single Precision)		
2	VAR_INPUT	iRef	Word[Signed]		
3	VAR_INPUT	fTs	FLOAT (Single Precision)		
4	VAR_INPUT	fRelayD	FLOAT (Single Precision)		
5	VAR_INPUT	fHyst	FLOAT (Single Precision)		
6	VAR_INPUT	fNomPådrag	FLOAT (Single Precision)		
7	VAR_INPUT	fN_filter	FLOAT (Single Precision)		
8	VAR_INPUT	bUseI	Bit		
9	VAR_INPUT	bUseD	Bit		
10	VAR_INPUT	bStart	Bit		
11	VAR_INPUT	bReset	Bit		
12	VAR_INPUT	iMinHalfCycles	FLOAT (Single Precision)		
13					
14	VAR_OUTPUT	fManOut	FLOAT (Single Precision)		
15	VAR_OUTPUT	fKp_out	FLOAT (Single Precision)		
16	VAR_OUTPUT	fKi_out	FLOAT (Single Precision)		
17	VAR_OUTPUT	fKd_out	FLOAT (Single Precision)		
18	VAR_OUTPUT	fTf_out	FLOAT (Single Precision)		
19	VAR_OUTPUT	fNom_out	FLOAT (Single Precision)		
20	VAR_OUTPUT	fKk	FLOAT (Single Precision)		
21	VAR_OUTPUT	fTk	FLOAT (Single Precision)		
22	VAR	bRunning	Bit		
23	VAR_OUTPUT	bDone	Bit		
24					
25	VAR	fError	FLOAT (Single Precision)		
26	VAR	fError_prev	FLOAT (Single Precision)		
27	VAR	fPeakHigh	FLOAT (Single Precision)		
28	VAR	fPeakLow	FLOAT (Single Precision)		
29	VAR	fAmplitude	FLOAT (Single Precision)		
30	VAR	fPeriod	FLOAT (Single Precision)		
31	VAR	fLastCrossTime	FLOAT (Single Precision)		
32	VAR	fCycleTimer	FLOAT (Single Precision)		
33	VAR	iHalfCycles	FLOAT (Single Precision)		
34	VAR	bRelayState	Bit		
35	VAR	bInitDone	Bit		
36	VAR	fTd_temp	FLOAT (Single Precision)		
37	VAR	fRef	FLOAT (Single Precision)		
38					
39	VAR_INPUT	bAbort	Bit		
40	VAR_INPUT	bEmergency	Bit		
41	VAR	bStartLatch	Bit		
42	VAR	fMeaFilt	FLOAT (Single Precision)		
43	VAR	bWasManual	Bit		

Label
Data Name : FO_VNT
Function/FB Label Setting

	Class	Label Name	Data Type	Constant	Comment
1	VAR_INPUT	fPID_in	FLOAT (Single Precision)		
2	VAR_INPUT	FO	Bit		
3	VAR_OUTPUT	FO_Out	Word[Signed]		
4	VAR_OUTPUT	VNT_Out	Word[Signed]		
5	VAR	iPID_in	Word[Signed]		
6	VAR_CONSTANT	input	Bit	TRUE	
7	VAR	bTransisjon	Bit		
8	VAR	iFO_hold	Word[Signed]		
9	VAR	FO_prev	Bit		

Label
Data Name : FRA_MASTER
Function/FB Label Setting

	Class	Label Name	Data Type	Constant	Comment
1					
2					
3	VAR_OUTPUT	Manuell	Bit		
4	VAR_OUTPUT	organ	Bit		
5	VAR_OUTPUT	ventil_1	Bit		
6	VAR_OUTPUT	ventil_2	Bit		
7	VAR_OUTPUT	ventil_3	Bit		
8					
9					
10	VAR_OUTPUT	Kp	FLOAT (Single Precision)		
11	VAR_OUTPUT	Ki	FLOAT (Single Precision)		
12	VAR_OUTPUT	Kd	FLOAT (Single Precision)		
13	VAR_OUTPUT	Tf	FLOAT (Single Precision)		
14					
15	VAR_OUTPUT	fManOutput	FLOAT (Single Precision)		
16	VAR_OUTPUT	iReferanse	Word[Signed]		
17	VAR_OUTPUT	bResetAlarm	Bit		
18	VAR_OUTPUT	iRateSek	Word[Signed]		
19					
20	VAR_OUTPUT	bStartAuto	Bit		
21	VAR_OUTPUT	bAbortAuto	Bit		
22	VAR_OUTPUT	bUseI	Bit		
23	VAR_OUTPUT	bUseD	Bit		
24	VAR_OUTPUT	fNomPådrag	FLOAT (Single Precision)		
25	VAR_OUTPUT	fKff	FLOAT (Single Precision)		
26	VAR_OUTPUT	bForover	Bit		
27	VAR_OUTPUT	bStart	Bit		
28	VAR_OUTPUT	bStopp	Bit		
29					
30	VAR	fKp_hmi	FLOAT (Single Precision)		
31	VAR	fKi_hmi	FLOAT (Single Precision)		
32	VAR	fKd_hmi	FLOAT (Single Precision)		
33	VAR	fTf_hmi	FLOAT (Single Precision)		

Label
Data Name : FRA_TANK
Function/FB Label Setting

5/3/2026

	Class	Label Name	Data Type	Constant	Comment
1	VAR	fAlpha	FLOAT (Single Precision)		
2	VAR	tank_p_hmi	FLOAT (Single Precision)		
3	VAR	tank_p_filt	FLOAT (Single Precision)		
4	VAR	bFirstRun	Bit		
5	VAR_OUTPUT	tank_p	FLOAT (Single Precision)		
6	VAR	tank	Word[Signed]		
7	VAR	ftank	FLOAT (Single Precision)		
8	VAR	fFlow_hmi	FLOAT (Single Precision)		
9	VAR_OUTPUT	fFlowLmin	FLOAT (Single Precision)		
10	VAR	fLastValid	FLOAT (Single Precision)		
11	VAR	fhmi	FLOAT (Single Precision)		
12	VAR	bHeivert	Bit		
13	VAR	iHeivertCount	Word[Signed]		
14	VAR	fDelta	FLOAT (Single Precision)		
15	VAR	fhmi_prev	FLOAT (Single Precision)		
16	VAR	tank_p_raw	FLOAT (Single Precision)		

	Class	Label Name	Data Type	Constant	Comment
1	VAR_INPUT	rInn	Word[Signed]		
2	VAR_INPUT	maksverdi	Word[Signed]		
3	VAR_OUTPUT	ProsentUt	Word[Signed]		
4	VAR_INPUT	offset	Word[Signed]		
5	VAR	Temp32	Double Word[Signed]		

Data Name : OPPSTART
Function/FB Label Setting

	Class	Label Name	Data Type	Constant	Comment
1	VAR_OUTPUT	bKlar	Bit		
2	VAR_OUTPUT	bStartAlarm	Bit		
3	VAR_OUTPUT	bVentil1	Bit		
4	VAR_OUTPUT	bVentil2	Bit		
5	VAR_OUTPUT	bVentil3	Bit		
6	VAR_OUTPUT	iReferanse	Word[Signed]		
7	VAR_OUTPUT	bMan	Bit		
8	VAR_INPUT	fTank_p	FLOAT (Single Precision)		
9	VAR	bStartupTimer	Bit		
10	VAR_OUTPUT	bRunning	Bit		
11	VAR_INPUT	bStopp	Bit		
12	VAR_INPUT	bStart	Bit		
13	VAR	fSkanTid	FLOAT (Single Precision)		
14	VAR	iSkanTid	Word[Signed]		
15					
16					
17					
18					
19					
20	VAR	fKp_saved	FLOAT (Single Precision)		
21	VAR	fKi_saved	FLOAT (Single Precision)		
22	VAR	fKd_saved	FLOAT (Single Precision)		
23	VAR	fTf_saved	FLOAT (Single Precision)		

Label
Data Name : PID_Regulator
Function/FB Label Setting

5/3/2026

	Class	Label Name	Data Type	Constant	Comment
1	VAR_INPUT	referans	Word[Signed]		
2	VAR_INPUT	mea	FLOAT (Single Precision)		
3	VAR_INPUT	fKp	FLOAT (Single Precision)		
4	VAR_INPUT	fKi	FLOAT (Single Precision)		
5	VAR_INPUT	fKd	FLOAT (Single Precision)		
6	VAR_OUTPUT	fOut	FLOAT (Single Precision)		
7	VAR_OUTPUT	fP	FLOAT (Single Precision)		
8	VAR	fError	FLOAT (Single Precision)		
9	VAR_OUTPUT	fI	FLOAT (Single Precision)		
10	VAR_CONSTANT	fTi	FLOAT (Single Precision)	10	
11	VAR	fTs	FLOAT (Single Precision)		
12	VAR_OUTPUT	fD_term	FLOAT (Single Precision)		
13	VAR	fError_prev	FLOAT (Single Precision)		
14	VAR	fTracking	FLOAT (Single Precision)		
15	VAR_CONSTANT	fNom	FLOAT (Single Precision)	39.0	
16	VAR	bUseI	Bit		
17	VAR	bUseD	Bit		
18	VAR_INPUT	fTf	FLOAT (Single Precision)		
19	VAR_CONSTANT	fTd	FLOAT (Single Precision)	1.0	
20	VAR	fDfilter	FLOAT (Single Precision)		
21	VAR	fhmiOut	FLOAT (Single Precision)		
22	VAR_INPUT	fFwd	FLOAT (Single Precision)		
23	VAR	bUseFwd	Bit		
24	VAR_INPUT	bMan	Bit		
25	VAR	fRef	FLOAT (Single Precision)		
26	VAR	fKt	FLOAT (Single Precision)		
27	VAR	fManOutput	FLOAT (Single Precision)		
28	VAR_CONSTANT	fOutMax	FLOAT (Single Precision)	100.0	
29	VAR	bSaturated	Bit		
30	VAR_CONSTANT	fOutMin	FLOAT (Single Precision)	0.0	
31	VAR_CONSTANT	input	Bit	TRUE	
32	VAR_INPUT	ManOutput	FLOAT (Single Precision)		
33	VAR_INPUT	fKff	FLOAT (Single Precision)		
34	VAR_INPUT	bForover	Bit		
35	VAR	bUseI_prev	Bit		
36	VAR	bInitDone	Bit		
37	VAR	bMan_prev	Bit		
38	VAR	fDynMin	FLOAT (Single Precision)		
39	VAR	fBoostTimer	FLOAT (Single Precision)		
40	VAR	fBoostStart	FLOAT (Single Precision)		
41	VAR	fBoostRamp	FLOAT (Single Precision)		
42	VAR	fBoostEnd	FLOAT (Single Precision)		
43	VAR	fRampDownTimer	FLOAT (Single Precision)		
44	VAR	fLocalMin	FLOAT (Single Precision)		
45	VAR	bLargeDownStep	Bit		
46	VAR	fRampFrac	FLOAT (Single Precision)		
47	VAR	fRampFloor	FLOAT (Single Precision)		
48	VAR	fRampDownStart	FLOAT (Single Precision)		
49	VAR	bHoldHigh	Bit		
50	VAR	iRef_prev	Word[Signed]		
51	VAR	fStableTimer	FLOAT (Single Precision)		
52	VAR	fMea_prev	FLOAT (Single Precision)		
53	VAR	bRampActive	Bit		
54	VAR	fI_rampStart	FLOAT (Single Precision)		
55	VAR	fI_rampProgress	FLOAT (Single Precision)		
56	VAR	bRampNomActive	Bit		
57	VAR	fT_smooth	FLOAT (Single Precision)		

Label
Data Name : Rampe
Function/FB Label Setting

5/3/2026

	Class	Label Name	Data Type	Constant	Comment
1	VAR_INPUT	OldValue	FLOAT (Single Precision)		
2	VAR_INPUT	NewValue	FLOAT (Single Precision)		
3	VAR	RampStep	FLOAT (Single Precision)		
4	VAR_OUTPUT	RampOut	FLOAT (Single Precision)		
5	VAR	rStartValue	FLOAT (Single Precision)		
6	VAR	rProgress	FLOAT (Single Precision)		
7	VAR	rLastTarget	FLOAT (Single Precision)		
8	VAR	rT_smooth	FLOAT (Single Precision)		
9	VAR_INPUT	iRateSek	Word[Signed]		
10	VAR	fRateSek	FLOAT (Single Precision)		
11	VAR	fTs	FLOAT (Single Precision)		

Data Name : TIL_MASTER

Function/FB Label Setting

	Class	Label Name	Data Type	Constant	Comment
1					
2					
3					
4					
5	VAR_INPUT	tank_p	FLOAT (Single Precision)		
6					
7	VAR_INPUT	bAlarmCritLow	Bit		
8	VAR_INPUT	bAlarmLow	Bit		
9	VAR_INPUT	bAlarmHigh	Bit		
10	VAR_INPUT	bAlarmCritHigh	Bit		
11	VAR_INPUT	bEmergency	Bit		
12	VAR	iAlarmWord	Word[Signed]		
13					
14	VAR_INPUT	fPådragUt	FLOAT (Single Precision)		
15	VAR_INPUT	fP	FLOAT (Single Precision)		
16	VAR_INPUT	fI	FLOAT (Single Precision)		
17	VAR_INPUT	fD_term	FLOAT (Single Precision)		
18	VAR	iP	Word[Signed]		
19	VAR	iI	Word[Signed]		
20	VAR	iD	Word[Signed]		
21	VAR_INPUT	fo_flow	Word[Signed]		
22					
23	VAR_INPUT	auto_p	FLOAT (Single Precision)		
24	VAR_INPUT	auto_i	FLOAT (Single Precision)		
25	VAR_INPUT	auto_d	FLOAT (Single Precision)		
26	VAR_INPUT	auto_tf	FLOAT (Single Precision)		
27					
28	VAR	iauto_p	Word[Signed]		
29	VAR	iauto_i	Word[Signed]		
30	VAR	iauto_d	Word[Signed]		
31	VAR	iauto_tf	Word[Signed]		
32	VAR	iStatus	Word[Signed]		
33	VAR_INPUT	bRunning	Bit		
34	VAR_INPUT	bKlar	Bit		
35					
36	VAR_INPUT	fFlowLmin	FLOAT (Single Precision)		
37	VAR	iFlowLmin	Word[Signed]		

Data Name : TIL_TANK
Function/FB Label Setting

	Class	Label Name	Data Type	Constant	Comment
1	VAR_INPUT	input	Bit		
2	VAR_INPUT	vnt_flow	Word[Signed]		
3	VAR_INPUT	fo_flow	Word[Signed]		

Label
Data Name : VERDIENDRING
Function/FB Label Setting

5/3/2026

	Class	Label Name	Data Type	Constant	Comment
1	VAR	Rampe_1	Rampe		
2	VAR_INPUT	Settpunkt_Kp	FLOAT (Single Precision)		
3	VAR_INPUT	Settpunkt_Ki	FLOAT (Single Precision)		
4	VAR_INPUT	Settpunkt_Kd	FLOAT (Single Precision)		
5	VAR_OUTPUT	Kp	FLOAT (Single Precision)		
6	VAR_OUTPUT	Ki	FLOAT (Single Precision)		
7	VAR_OUTPUT	Kd	FLOAT (Single Precision)		
8	VAR_OUTPUT	referanse	Word[Signed]		
9	VAR_INPUT	Settpunkt_referanse	Word[Signed]		
10	VAR	fSettpunkt_referanse	FLOAT (Single Precision)		
11	VAR	fReferanse	FLOAT (Single Precision)		
12	VAR	Rampe_2	Rampe		
13	VAR	Rampe_3	Rampe		
14	VAR	Rampe_4	Rampe		
15	VAR_INPUT	iRateSek	Word[Signed]		
16	VAR	Rampe_5	Rampe		
17	VAR_OUTPUT	fKff	FLOAT (Single Precision)		
18	VAR_INPUT	fSettpunkt_Kff	FLOAT (Single Precision)		
19	VAR_INPUT	fSettpunkt_NomPådrag	FLOAT (Single Precision)		
20	VAR	Rampe_6	Rampe		
21	VAR_OUTPUT	fNomPådrag	FLOAT (Single Precision)		